ECE3411 – Fall 2025

Lecture 12

# EEPROM

**Sung Yeul Park**
Department of Electrical & Computer Engineering
University of Connecticut
Email: sung_yeul.park@uconn.edu

Copied from Lecture 5c, ECE3411 – Fall 2015, by
Marten van Dijk and Syed Kamran Haider

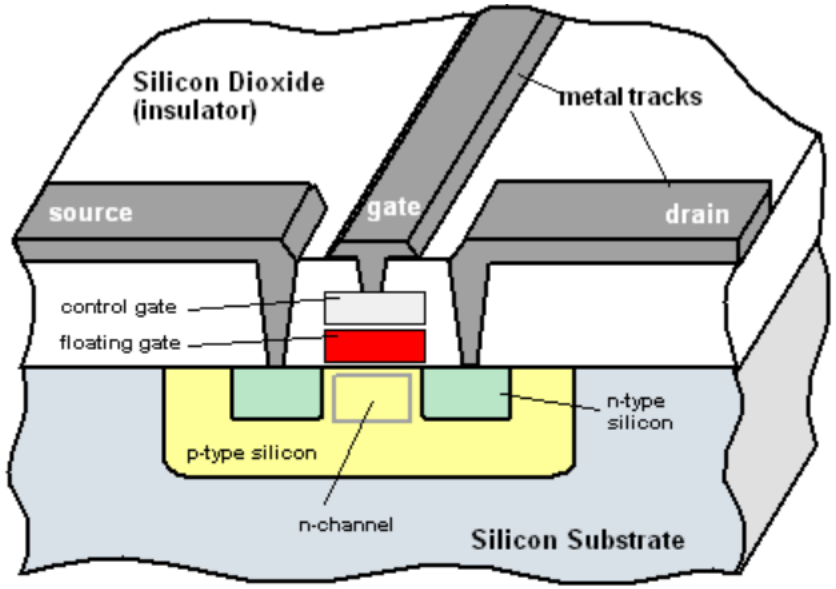
Adapted from John Chandy ECE3411 – Fall 2024

# EEPROM: Electrically Erasable Programmable ROM

1. What is EEPROM?
- EEPROM stands for Electrically Erasable Programmable Read-Only Memory.
- It is a non-volatile memory, which means data is retained even after power is turned off.
- EEPROM is useful for storing settings, calibration values, or user preferences that must survive resets or shutdowns.

2. Types of EEPROM
- Internal EEPROM: Built into AVR, 512B – 4kB in size
- External EEPROM: I2C or SPI, 24LC256, AT24C02

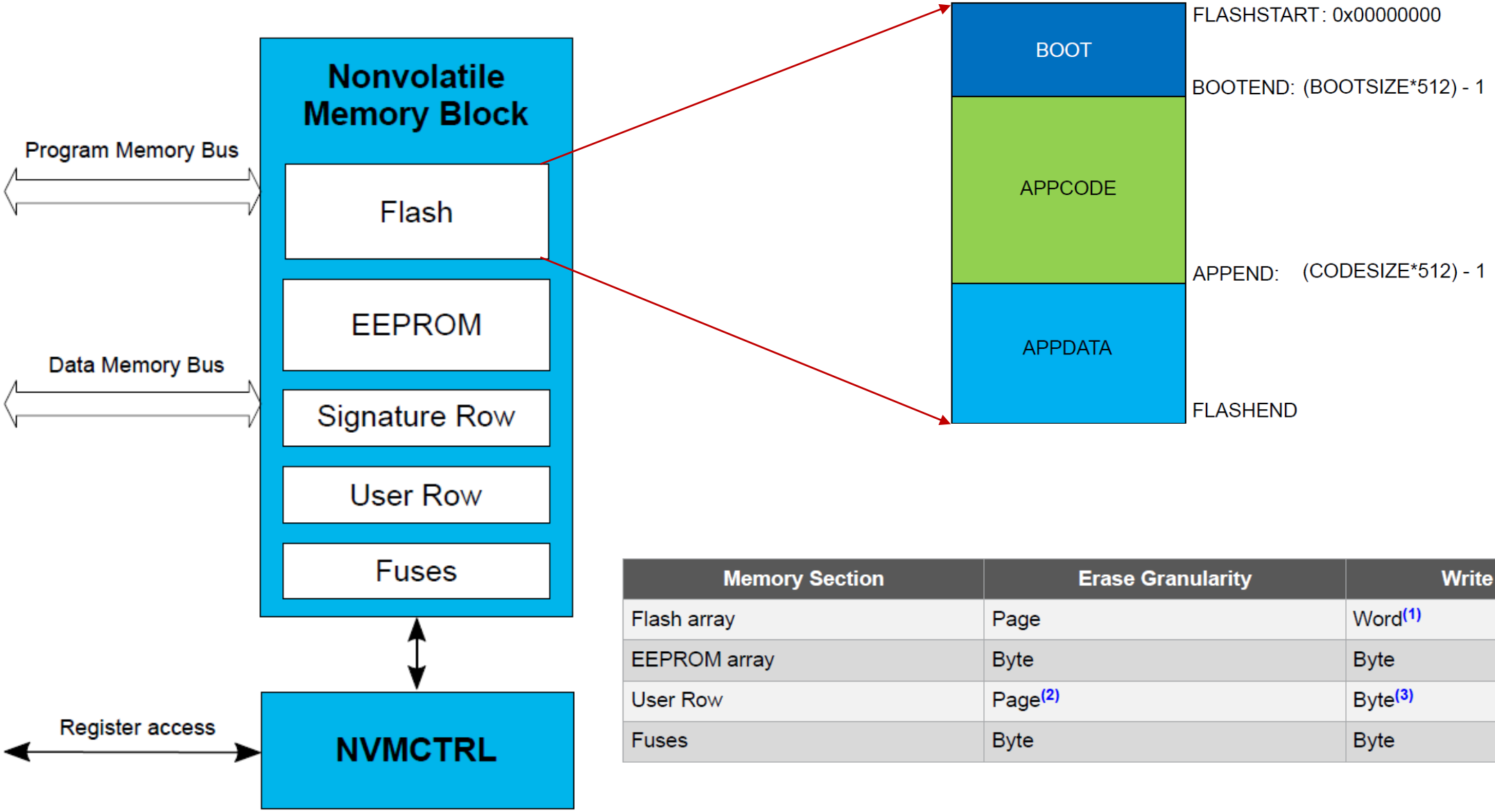
3. Applications
- Storing device configuration
- Logging sensor values occasionally
- Saving user settings across power cycles




**The Floating Gate Transistor**

EEPROM and flash memory cells use a transistor with a floating gate that holds a charge. When charged, the action of the control gate is impeded, and the charged/uncharged state determines the 0 or 1 content of the bit.

# NVMCTRL Block Diagram




| Memory Section | Erase Granularity | Write Granularity |
|---|---|---|
| Flash array | Page | Word(1) |
| EEPROM array | Byte | Byte |
| User Row | Page(2) | Byte(3) |
| Fuses | Byte | Byte |

# EEPROM: Electrically Erasable Programmable ROM

- AVR128DB contains 512B of EEPROM memory – separate from flash and SRAM
- Data in EEPROM remains if you turn power off and back on or even if you pull the chip out of the board
- Allows you to keep data that is retained even when program is updated
- EEPROM: write one byte in 70 µs, byte erase in 10 ms
- EEPROM can be erased/written one byte at a time
- Flash: write one byte in 70 µs, chip erase in 11 ms
- Flash must be page erased before writing (e.g., 64-byte chunk)
- RAM: write in 1 cycle (62.5 ns)

# NVMCTRL.CTRLA: Nonvolatile Memory Controller

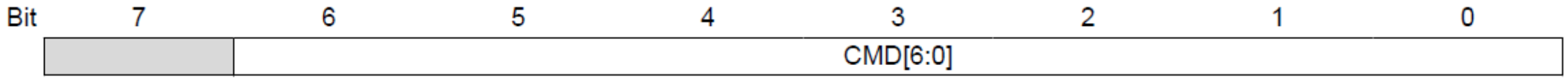
11.3.2.3.4 EEPROM Write Mode
The EEPROM Write (EEWR) mode enables the EEPROM array for writing operations. Several writes can be done while the EEWR mode is enabled in the NVMCTRL.CTRLA register. When the EEWR mode is enabled, writes with the ST* instructions will be performed one byte at a time. When writing the EEPROM, the CPU will continue executing code. If a new load/store operation is started before the EEPROM erase/write is completed, the CPU will be halted. Erasing its content is necessary before performing a write to an address.

11.3.2.3.5 EEPROM Erase/Write Mode
The EEPROM Erase/Write (EEERWR) mode enables the EEPROM array for the erase operation directly followed by a write operation. Several erase/writes can be done while the EEERWR mode is enabled in the NVMCTRL.CTRLA register. When the EEERWR mode is enabled, writes with the ST* instructions are performed one byte at a time. When writing/erasing the EEPROM, the CPU will continue executing code. If a new load or store instruction is started before the erase/write is completed, the CPU will be halted.

11.3.2.3.6 EEPROM Byte Erase Mode
The EEPROM Byte Erase (EEBER) mode allows each write to the memory array to erase the selected byte. An erased byte always reads back 0xFF, regardless of the value written to the EEPROM address. When erasing the EEPROM, the CPU can continue running from the Flash. If the CPU starts an erase or write operation while the EEPROM is busy, the CPU will be halted until finishing the current operation.

11.3.2.3.7 EEPROM Multi-Byte Erase Mode
The EEPROM Multi-Byte Erase (EEMBERn) mode allows erasing several bytes in one operation. When enabling the EEMBERn mode, it is possible to select between erasing 2, 4, 8, 16, or 32 bytes in one operation. The LSbs of the address are ignored when defining which EEPROM locations are erased. For example, while doing an 8-byte erase, addressing any byte in the 0x18 - 0x1F range will result in erasing the entire range of bytes.

# NVMCTRL.CTRLA: Nonvolatile Memory Controller




A change from one command to another should always go through a No command(`NOCMD`) or No operation(`NOOP`) .

| Value | Name | Description |
|---|---|---|
| 0x00 | NOCMD | No command |
| 0x01 | NOOP | No operation |
| 0x02 | FLWR | Flash Write Enable |
| 0x08 | FLPER | Flash Page Erase Enable |
| 0x09 | FLMPER2 | Flash 2-page Erase Enable |
| 0x0A | FLMPER4 | Flash 4-page Erase Enable |
| 0x0B | FLMPER8 | Flash 8-page Erase Enable |
| 0x0C | FLMPER16 | Flash 16-page Erase Enable |
| 0x0D | FLMPER32 | Flash 32-page Erase Enable |
| 0x12 | EEWR | EEPROM Write Enable |
| 0x13 | EEERWR | EEPROM Erase and Write Enable |
| 0x18 | EEBER | EEPROM Byte Erase Enable |
| 0x19 | EEMBER2 | EEPROM 2-byte Erase Enable |
| 0x1A | EEMBER4 | EEPROM 4-byte Erase Enable |
| 0x1B | EEMBER8 | EEPROM 8-byte Erase Enable |
| 0x1C | EEMBER16 | EEPROM 16-byte Erase Enable |
| 0x1D | EEMBER32 | EEPROM 32-byte Erase Enable |
| 0x20 | CHER | Erase Flash and EEPROM. EEPROM is skipped if EESAVE fuse is set. (UPDI access only.) |
| 0x30 | EECHER | Erase EEPROM |
| Other | - | Reserved |

# NVMCTRL.ADDR

NVMCTRL.ADDR0, NVMCTRL.ADDR1 and NVMCTRL.ADDR2 represent the 24-bit value NVMCTRL.ADDR.

The low byte [7:0] (suffix 0) is accessible at the original offset.

The high byte [15:8] (suffix 1) can be accessed at offset +0x01.

The extended byte [23:16] (suffix 2) can be accessed at offset +0x02.

| Bit | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|
| | ADDR[23:16] | | | | | | | |
| Access | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Reset | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Bit | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|---|
| | ADDR[15:8] | | | | | | | |
| Access | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Reset | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | ADDR[7:0] | | | | | | | |
| Access | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Reset | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Bits 23:0 – ADDR[23:0]** Address

The Address register contains the address of the last memory location that has been accessed. Only the number of bits required to access the memory is used.

# NVMCTRL.DATA

### 11.5.6 Data

The NVMCTRL.DATAL and NVMCTRL.DATAH register pair represents the 16-bit value, NVMCTRL.DATA.

The low byte [7:0] (suffix L) is accessible at the original offset.

The high byte [15:8] (suffix H) can be accessed at offset + `0x01`.

| Bit | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|---|
| | DATA[15:8] | | | | | | | |
| Access | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Reset | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | DATA[7:0] | | | | | | | |
| Access | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |
| Reset | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

**Bits 15:0 – DATA[15:0]** Data Register

The Data register will contain the last read value from Flash, EEPROM, or NVMCTRL. For EEPROM access, only DATA[7:0] is used.

# NVMCTRL.STATUS

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | | ERROR[2:0] | | | | | EEBUSY | FBUSY |
| Access | | R/W | R/W | R/W | | | R | R |
| Reset | | 0 | 0 | 0 | | | 0 | 0 |

**Bits 6:4 – ERROR[2:0]** Error Code

The Error Code bit field will show the last error occurring. This bit field can be cleared by writing it to '0'.

| Value | Name | Description |
|---|---|---|
| 0x0 | NONE | No error |
| 0x1 | INVALIDCMD | The selected command is not supported |
| 0x2 | WRITEPROTECT | Attempt to write a section that is protected |
| 0x3 | CMDCOLLISION | A new write/erase command was selected while a write/erase command is already ongoing |
| Other | — | Reserved |

**Bit 1 – EEBUSY** EEPROM Busy

This bit will read '1' when an EEPROM programming operation is ongoing.

**Bit 0 – FBUSY** Flash Busy

This bit will read '1' when a Flash programming operation is ongoing.

# EEPROM writing process

- Check that the EEPROM is not busy – `EEBUSY` flag in `NVMCTRL.STATUS` register
- Write the SPM Instruction protection key (0x9D) to the `CPU.CCP` register (Datasheet 7.4.6.2)
- Write the erase/write command to the `NVMCTRL.CTRLA` within the next four instructions
- Write the data to the EEPROM address
- Write the `NONE` or `NOOP` command to the `NVMCTRL.CTRLA` register

**7.4.6.2 Sequence for Execution of Self-Programming**

To execute self-programming (the execution of writes to the NVM controller's command register), the following steps are required:

1. The software temporarily enables self-programming by writing the SPM signature to the CCP register (CPU.CCP).
2. Within four instructions, the software must execute the appropriate instruction. The protected change is immediately disabled if the CPU performs accesses to the Flash, NVMCTRL, or EEPROM, or if the `SLEEP` instruction is executed.

# EEPROM Write

```
void eeprom_write_byte (uint8_t * address, uint8_t data)
{
        // Wait for EEPROM to be ready
        while (NVMCTRL.STATUS & NVMCTRL_EEBUSY_bm);
        // Set command: Erase + Write
        CPU_CCP = CCP_SPM_gc;
        NVMCTRL.CTRLA = NVMCTRL_CMD_EEERWR_gc;
        // Write the byte to the EEPROM memory space
        *((uint8_t*)EEPROM_START + address) = data;
       // Wait until write completes
        while (NVMCTRL.STATUS & NVMCTRL_EEBUSY_bm);
        CPU_CCP = CCP_SPM_gc; // Clear command
        NVMCTRL.CTRLA = NVMCTRL_CMD_NONE_gc;
}
```

There can be no more than 4 cycles between these two instructions. HW automatically disables interrupts.

# EEPROM Read

```
uint8_t eeprom_read_byte(uint8_t* address)
{
        /* read the data */
        uint8_t data;
        data = *((uint8_t*)EEPROM_START + address);
}
```

# Sample Code

```c
#define F_CPU 16000000UL  // External 16 MHz clock

#include <avr/io.h>
#include <util/delay.h>
#include <avr/eeprom.h>

// EEPROM address
#define EEPROM_ADDR 0x00

void CLK_init(void) {

CPU_CCP = CCP_IOREG_gc;
CLKCTRL.XOSCHFCTRLA = CLKCTRL_FRQRANGE_16M_gc |
CLKCTRL_ENABLE_bm;
CPU_CCP = CCP_IOREG_gc;
CLKCTRL.MCLKCTRLA = CLKCTRL_CLKSEL_EXTCLK_gc;

// Wait for clock to stabilize
_delay_ms(100);
}


void PORT_init(void) {
// Set PB3 as output (LED)
PORTB.DIRSET = PIN3_bm;
}
```

```c
int main(void) {
CLK_init();     // Initialize clock
PORT_init();    // Initialize ports

uint8_t write_data = 0x5A;
uint8_t read_data;

// Write to EEPROM
eeprom_write_byte((uint8_t*)EEPROM_ADDR, write_data);
_delay_ms(10);  // Wait for write to complete

// Read from EEPROM
read_data = eeprom_read_byte((uint8_t*)EEPROM_ADDR);

// Check if read matches written value
if (read_data == write_data) {
PORTB.OUTCLR = PIN3_bm;  // Turn on LED at PB2
} else {
PORTB.OUTSET = PIN3_bm;  // Turn off LED
}

while (1) {
// Loop forever
}
}
```

# Lab Practice #11 : Password setting using EEPROM

- Step 1: EEPROM Read and Display
  - On startup, read the stored password from EEPROM.
  - Display it via USART3 (for debugging only).
- Step 2: Password Verification
  - Prompt user via USART3 to enter a 4-character password.
  - Compare input with EEPROM-stored password.
  - If matched, turn on PB3 LED and send “Access Granted” via USART3.
  - If not matched, blink PB3 LED 3 times and send “Access Denied”.
- Step 3: Password Update via Button.
  - When PB5 is pressed, prompt user to enter a new password via USART3.
  - Store the new password in EEPROM.
  - Confirm update via USART3.